



Trailhead



[Apex Triggers \(/content/learn/modules/apex_triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)



Bulk Apex Triggers

Bulk Apex Triggers

Learning Objectives

After completing this unit, you'll be able to:

- Write triggers that operate on collections of sObjects.
- Write triggers that perform efficient SOQL and DML operations.

Bulk Trigger Design Patterns

Apex triggers are optimized to operate in bulk. We recommend using bulk design patterns for processing records in triggers. When you use bulk design patterns, your triggers have better performance, consume fewer server resources, and are less likely to exceed platform limits.

The benefit of bulkifying your code is that bulkified code can process large numbers of records efficiently and run within governor limits on the Salesforce Platform. These governor limits are in place to ensure that runaway code doesn't monopolize resources on the multitenant platform.

The following sections demonstrate the main ways of bulkifying your Apex code in triggers: operating on all records in the trigger, and performing SOQL and DML on collections of sObjects instead of single sObjects. The SOQL and DML bulk best practices apply to any Apex code, including SOQL and DML in classes. The examples given are based on triggers and use the `Trigger.new` [copy](#) context variable.

Operating on Record Sets

Let's first look at the most basic bulk design concept in triggers. Bulkified triggers operate on all sObjects in the trigger context. Typically, triggers operate on one record if the action that fired the trigger originates from the user interface. But if the origin action was bulk DML or the API, the trigger operates on a record set rather than c





trigger to fire. This trigger doesn't work on a full record set when multiple records are inserted in the same transaction. A bulkified version is shown in the next example.

[Apex Triggers \(/content/learn/modules/apex_triggers?](#)

[trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\).](#)

```
1 trigger MyTriggerNotBulk on Account(before insert) {
2     Account a = Trigger.new[0];
3     a.Description = 'New description';
4 }
```



Bulk Apex Triggers

This example (`MyTriggerBulk`) is a modified version of `MyTriggerNotBulk` . It uses a `for` loop to iterate over all available sObjects. This loop works if `Trigger.new` contains one sObject or many sObjects.

```
1 trigger MyTriggerBulk on Account(before insert) {
2     for(Account a : Trigger.new) {
3         a.Description = 'New description';
4     }
5 }
```



Performing Bulk SOQL

SOQL queries can be powerful. You can retrieve related records and check a combination of multiple conditions in one query. By using SOQL features, you can write less code and make fewer queries to the database. Making fewer database queries helps you avoid hitting query limits, which are 100 SOQL queries for synchronous Apex or 200 for asynchronous Apex.

The following trigger (`SoqlTriggerNotBulk`) shows a SOQL query pattern to avoid. The example makes a SOQL query inside a `for` loop to get the related opportunities for each account, which runs once for each Account sObject in `Trigger.new` . If you have a large list of accounts, a SOQL query inside a `for` loop could result in too many SOQL queries. The next example shows the recommended approach.

```
1 trigger SoqlTriggerNotBulk on Account(after update) {
2     for(Account a : Trigger.new) {
3         // Get child records for each account
4         // Inefficient SOQL query as it runs once for each
5         account!
```





```
}
```

Apex Triggers (/content/learn/modules/apex_triggers?

The following example ([SoqlTriggerBulk](#)) is a modified version of the previous one ([trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer](#)) and shows a best practice for running SOQL queries. The SOQL query does the heavy lifting and is called once outside the main loop.

Bulk Apex Triggers

- The SOQL query uses an inner query—(`SELECT Id FROM Opportunities`)—to get related opportunities of accounts.
- The SOQL query is connected to the trigger context records by using the `IN` clause and binding the `Trigger.new` variable in the `WHERE` clause—`WHERE Id IN :Trigger.new`. This `WHERE` condition filters the accounts to only those records that fired this trigger.

Combining the two parts in the query results in the records we want in one call: the accounts in this trigger with the related opportunities of each account.

After the records and their related records are obtained, the `for` loop iterates over the records of interest by using the collection variable—in this case, `acctsWithOpps`. The collection variable holds the results of the SOQL query. That way, the `for` loop iterates only over the records we want to operate on. Because the related records are already obtained, no further queries are needed within the loop to get those records.

```
1  trigger SoqlTriggerBulk on Account(after update) {
2      // Perform SOQL query once.
3      // Get the accounts and their related opportunities.
4      List<Account> acctsWithOpps =
5          [SELECT Id, (SELECT Id,Name,CloseDate FROM Opportunities)
6            FROM Account WHERE Id IN :Trigger.new];
7      // Iterate over the returned accounts
8      for(Account a : acctsWithOpps) {
9          Opportunity[] relatedOpps = a.Opportunities;
10         // Do some other processing
11     }
12 }
```



Alternatively, if you don't need the account parent records, you can retrieve only the opportunities that are related to the accounts within this trigger context. This list is specified in the `WHERE` clause by matching the `AccountId` field of the



```

1  trigger SoqlTriggerBulk on Account(after update) {
2      // Perform SOQL query once.
3      // Get the related opportunities for the accounts in this
4      trigger.
5      List<Opportunity> relatedOpps = [SELECT Id,Name,CloseDate
6      FROM Opportunity
7      WHERE AccountId IN :Trigger.new];
8      // Iterate over the related opportunities
9      for(Opportunity opp : relatedOpps) {
10         // Do some other processing
11     }
12 }

```



[Apex Triggers \(/content/learn/modules/apex_triggers?](https://trailhead.salesforce.com/content/learn/modules/apex_triggers/apex_triggers_bulk?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

[trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/apex_triggers/apex_triggers_bulk?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Bulk Apex Triggers

You can reduce the previous example in size by combining the SOQL query with the `for` loop in one statement: the SOQL `for` loop. Here is another version of this bulk trigger using a SOQL `for` loop.

```

1  trigger SoqlTriggerBulk on Account(after update) {
2      // Perform SOQL query once.
3      // Get the related opportunities for the accounts in this
4      trigger,
5      // and iterate over those records.
6      for(Opportunity opp : [SELECT Id,Name,CloseDate FROM
7      Opportunity
8      WHERE AccountId IN :Trigger.new]) {
9          // Do some other processing
10     }
11 }

```



Triggers execute on batches of 200 records at a time. So if 400 records cause a trigger to fire, the trigger fires twice, once for each 200 records. For this reason, you don't get the benefit of SOQL `for` loop record batching in triggers, because triggers batch up records as well. The SOQL `for` loop is called twice in this example, but a standalone SOQL query would also be called twice. However, the SOQL `for` loop still looks more elegant than iterating over a collection variable!



This trigger ([DmlTriggerNotBulk](#)) performs an update call inside a [for](#) loop that iterates over related opportunities. If certain conditions are met, the trigger updates the opportunity description. In this example, the update statement is inefficiently called once for each opportunity. If a bulk account update operation fired the trigger, there can be many updates. If each account has one or two opportunities, we can easily end up with over 150 opportunities. The DML statement limit is 150 calls.

```
1 trigger DmlTriggerNotBulk on Account(after update) {
2     // Get the related opportunities for the accounts in this
3     trigger.
4     List<Opportunity> relatedOpps = [SELECT Id,Name,Probability
5     FROM Opportunity
6     WHERE AccountId IN :Trigger.new];
7     // Iterate over the related opportunities
8     for(Opportunity opp : relatedOpps) {
9         // Update the description when probability is greater
10        // than 50% but less than 100%
11        if ((opp.Probability >= 50) && (opp.Probability < 100))
12        {
13            opp.Description = 'New description for
14            opportunity.';
15            // Update once for each opportunity -- not
16            efficient!
17            update opp;
18        }
19    }
20 }
```



This next example ([DmlTriggerBulk](#)) shows how to perform DML in bulk efficiently with only one DML call on a list of opportunities. The example adds the Opportunity sObject to update to a list of opportunities ([oppsToUpdate](#)) in the loop. Next, the trigger performs the DML call outside the loop on this list after all opportunities have been added to the list. This pattern uses only one DML call regardless of the number of sObjects being updated.

```
1 trigger DmlTriggerBulk on Account(after update) {
2     // Get the related opportunities for the accounts in this
3     trigger.
4
```





```

10     for(Opportunity opp : relatedOpps) {
11         // Update the description when probability is greater
12         // than 50%
13         if(opp.Probability > 50) {
14             opp.Description = 'New description for
15             opportunity.';
16             oppsToUpdate.add(opp);
17         }
18     }
19     // Perform DML on a collection
20     update oppsToUpdate;
21 }

```

[Apex Triggers \(/content/learn/modules/apex_triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

Bulk Apex Triggers

Bulk Design Pattern in Action: Example of Getting Related Records

Let's apply the design patterns you've learned by writing a trigger that accesses accounts' related opportunities. Modify the trigger example from the previous unit for the

`AddRelatedRecord` trigger. The `AddRelatedRecord` trigger operates in bulk, but the one from the previous unit isn't as efficient as it could be because it iterates over all `Trigger.New` sObject records. This next example modifies both the trigger code and the SOQL query to get only the records of interest and then iterate over those records. If you haven't created this trigger, don't worry—you can create it in this section.

Let's start by reviewing the requirements for the `AddRelatedRecord` trigger. The trigger fires after accounts are inserted or updated. The trigger adds a default opportunity for every account that doesn't already have an opportunity.

First of all, newly-inserted accounts never have a default opportunity, so we definitely need to add one. But for updated accounts, we need to determine whether they have a related opportunity or not. So let's separate how we process inserts and updates by using a `switch` statement on the `Trigger.operationType` context variable. And then keep track of the accounts we need to process with a `toProcess` variable. For example:

```

1  List<Account> toProcess = null;
2  switch on Trigger.operationType {
3      when AFTER_INSERT {
4          // do stuff

```





For all account inserts, we simply assign the new accounts to the `toProcess` `copy` `list`:
[Apex Triggers \(/content/learn/modules/apex_triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#).

Bulk Apex Triggers

```
1  when AFTER_INSERT {
2      toProcess = Trigger.New;
3  }
```



For updates, we must figure out which existing accounts in this trigger don't already have a related opportunity. Because this trigger is an after trigger, we can query the affected records from the database. Here's the SOQL statement, the results of which we assign to the `toProcess` `copy` `list`.

```
1  when AFTER_UPDATE {
2      toProcess = [SELECT Id,Name FROM Account
3                  WHERE Id IN :Trigger.New AND
4                  Id NOT IN (SELECT AccountId FROM Opportunity
5                  WHERE AccountId in :Trigger.New)];
6  }
```



We now use a `for` `copy` `loop` to iterate through the `toProcess` `copy` `list` of accounts and add a related default opportunity to `oppList` `copy` `list`. When we're finished, we bulk add the list of opportunities using the `insert` `copy` `DML` statement. Here's how to create or update the complete trigger.

1. If you've already created the `AddRelatedRecord` `copy` `trigger` in the previous unit, modify the trigger by replacing its contents with the following trigger. Otherwise, add the following trigger using the Developer Console and enter `AddRelatedRecord` `copy` `for` the trigger name.

```
1  trigger AddRelatedRecord on Account(after insert, after update) {
2      {
3          List<Opportunity> oppList = new List<Opportunity>();
4          // Add an opportunity for each account if it doesn't already
5          have one.
6          // Iterate over accounts that are in this trigger but that
7          don't have opportunities.
8          List<Account> toProcess = null;
9          switch on Trigger.operationType {
```





```

15         when AFTER_UPDATE {
16             toProcess = [SELECT Id,Name FROM Account
17                           WHERE Id IN :Trigger.New AND
18                           Trailmix__Creator__Id FROM
19                           Opportunity WHERE AccountId in :Trigger.New)];
20     }

```

[Apex Triggers \(/content/learn/modules/apex-triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

Bulk Apex Triggers

Show More

2. To test the trigger, create an account in the Salesforce user interface and name it

Lions and Cats .

3. In the Opportunities related list on the account's page, find the new opportunity

Lions and Cats . The trigger added the opportunity automatically!

Resources

- [Apex Developer Guide: Triggers \(https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_triggers.htm\)](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_triggers.htm).
- [Apex Developer Guide: Execution Governors and Limits \(https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_gov_limits.htm\)](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_gov_limits.htm).

GET READY

You'll be completing this unit in your own hands-on org. Click **Launch** to get started, or click the name of your org to choose a different one.

YOUR CHALLENGE

Create a Bulk Apex trigger

Create a bulkified Apex trigger that adds a follow-up task to an opportunity if its stage is Closed Won. Fire the Apex trigger after inserting or updating an opportunity.

- Create an Apex trigger:



- Subject : Follow Up Test Task
 - WhatId : the opportunity ID (associates the task with the opportunity)
- [Apex Triggers \(/content/learn/modules/apex_triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)
- Bulkify the Apex trigger so that it can insert or update 200 or more opportunities
- Bulk Apex Triggers**

Choose hands-on org

Koushik nelluri Salesforce CRM

Created on 5/9/2026



Launch

Check Challenge to Earn 500 Points

Learn

Badges
Trails
Trailmixes
Superbadges
Trailhead Academy
Career Paths

Certifications

Certifications
Maintain Certifications
Verify Certifications

Community

Trailblazer Community
Events
Agentblazers
Salesforce Developers
Salesforce Admins
Code of Conduct
Report Illegal Content (EU)

Extras

Trailhead Login
Sales Enablement
Trailblazer Store
Salesforce Help

Download Trailhead GO





[Apex Triggers \(/content/learn/modules/apex_triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/apex_triggers?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Bulk Apex Triggers